

Epiq Revenue-Leakage — Team Briefing

EVE Glyph Design · Anchored to Tobias Polly, polly consulting e.U. (17 July 2026) · Status: Propose-only

Where to go

Onboarding surface: eveglyphdesign.github.io/epiq-revenue-leakage

Repository: github.com/EVEglyphDesign/epiq-revenue-leakage

Open the surface, get connected, spin the model. Everything works off one repo.

What this is

A fork off of EVE — the EVE-side baseline. We use it to clear up the requirements and stand up the initial demo. That is the whole job right now.

- All Epiq data stays inside the Epiq environment. Demo data lives in Postgres, inside Epiq infrastructure. None of Epiq's data is handled outside it.
- No proprietary Epiq information in this repo. The client-confidential side is set up and governed by Tobias, walled off — we contribute through him.
- On approval, the demo becomes a repository build on Epiq infrastructure. The EVE fork copies back into the Epiq environment and we continue there.
- We do not own the capital asset — that is Epiq's.

The model in one line

Revenue leakage in enterprise services is correctable — but only if you catch it in the cycle it happens. This is a reasoning agent grounded in the client's own edit history, not another dashboard or narrative.

The pattern-analysis pipeline — who owns each stage

The live SAP → DataSphere feed comes first (Seshu + Satya); the engine is already running behind it, plus a validation gate before it opens in Teams. Full architecture in Document B.

#	Stage	What happens	Owner	Status
1	Live SAP → DataSphere pipeline	Stand up the live connection and pipeline all SAP data out via DataSphere — the feed everything else reads from	Seshu works with Satya	Build — top step
2	Source adapter	Crosses the boundary into Epiq's live system; presents edit history in one uniform shape	Tobias (points it)	Configure
3	Reasoning agent	Bounded Router → Executor → Synthesizer; plans, fetches, narrates in plain language	Tobias — reused unchanged	Live
4	Compute layer	Ephemeral DuckDB (+ graph if needed); spun up per session, torn down after	Tobias	Configure

pour le bien-être du peuple

#	Stage	What happens	Owner	Status
5	Model deployment	Azure AI Foundry model confirmed in-tenant, inside the network boundary	Tobias	Configure
6	Delivery (Teams)	Users ask in plain English, get answer + card; no SQL, no dashboard	Tobias — reused unchanged	Live
7	Observability	Langfuse records every step end-to-end; any answer replayable	Tobias — reused unchanged	Live
8	Revenue validation gate	Validate leakage answers against a known period — first target: the Fiori auto-billing patterns — then open in Teams	Tobias validates · Dany/PMO reviews	Pending

What Tobias owns to complete — the short list

The live SAP → DataSphere feed (Seshu + Satya) has to exist first — everything below reads from it. Everything client-side and confidential is Tobias's, walled off from this repo. To finish the retargeting from architecture to a live Revenue Leakage demo:

1. Point the source adapter at the live SAP → DataSphere edit-history feed once Seshu and Satya have it standing.
2. Stand up the ephemeral DuckDB layer (add the graph only if relationship questions demand it).
3. Confirm the model deployment and network boundary in Epiq's tenant.
4. Supply the revenue-specific model detail the architecture extract does not yet carry (the gap — cued in Document B): exact leakage measures / field rules (which post-booking edits count, thresholds); the finding schema the Synthesizer emits for revenue answers; the known-period validation set used to prove the answers.
5. Validate against a known period — first target the Fiori auto-billing patterns — then open it in Teams.

Steps 1–3 and 5 are configuration on the running engine. Step 4 is the only net-new input, and it comes from Tobias — nothing in this repo invents it.

What Seshu does — the live pipeline (top of the build)

Seshu works with Satya to stand up the live SAP → DataSphere connection and pipeline all the SAP data out via DataSphere at once. This is the feed everything else reads from — it is the first thing that has to exist, not a later step.

What Matt brings in — the Fiori draft-billing twin

Matt drops the draft billing (Fury) app into a billing-twin/ folder: table names, keys, relationships, schema exports/DDL, and the SAP data it depends on. This is a Fiori draft-billing process that runs auto-billing.

The leakage hypothesis lives here. We think there is probably leakage in the auto-billing, and that we can improve it — but we need to run the pattern analysis on it to confirm before we act. The goal is to twin Jackie's whole operation so the engine can check the auto-billing patterns against the live feed.

Getting connected (short version)

1. Free GitHub account (if you don't have one).
2. Send your GitHub username to the group to be added.
3. Clone the repo (or press . on the repo page for the browser editor).
4. Point your surface of choice at the folder (Cursor, VS Code + Copilot, Claude/ChatGPT, or the GitHub web editor).
5. Make changes → commit → push. That flows back to the shared model and the surface.